



Instructor: Date:

Learner: Regional Branch:

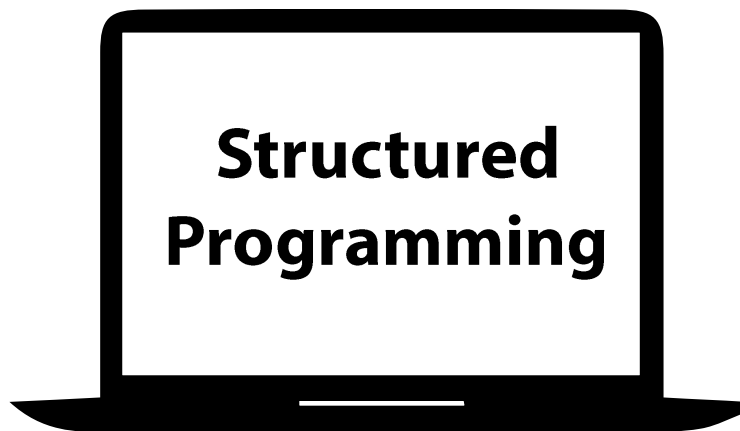
Program: Center:

CONTENT

- Let's Start!
- Let's Explore!
- Let's Practice!
- Extension Activity
- Target Words
- Learning Points

**Let's Start!**

The Origins of Structured Programming: The Beginning of Something Big

**Learning Outcome:**

Understand key information about the concepts, principles, and methodologies in simple spoken and written situations.
Talk about programming techniques, use cases, kinds, and sequences in simple spoken and written situations.

Ask your partner:

- What was the first software you used in your life?
- How do current software programs relate to the first ones created?

Can Do Statements:

- I can explain what structured programming is.
- I can talk about the role of Edsger Dijkstra in structured programming.
- I can obtain information about the principles of structured programming.





Let's Explore!

Mandy helps Kyle to understand structured programming.



Instructions: Read the text below.

- Kyle:** After reading this article, I feel like I wasted my time because I didn't understand anything.
- Mandy:** What are you reading about? Maybe I can help you.
- Kyle:** It's about structured programming. You know, history, Dijkstra, and the principles. I hope you can help me. Sometimes I need to ask for help.
- Mandy:** Sure! Before, computers weren't so complex. When scientists started to implement more functions, computers and software became more difficult to understand. So, in 1968, the scientist Edsger Dijkstra said something that changed programming forever. His role in programming is very important. Are you familiar with that name?
- Kyle:** Yes! Dijkstra stated that the "GO TO" statement wasn't beneficial for programming. Those statements were harmful because they made the logic of the operation more difficult to understand.
- Mandy:** That's right! I can see you know about this. Sometimes you have to take plenty of notes because those topics are vast. Well, do you need to study more?
- Kyle:** Yes Mandy. Before doing my presentation, I need to understand the concept of structured programming.
- Mandy:** Ok, just let me tell you I always have to make my best to explain this difficult topic. I'll try anyway. The structured programming refers to a set of principles or pre-established tasks and subroutines. This transfers tasks easier so, the understanding of software and its performance is also easier.
- Kyle:** I think I get it now! I usually need to research more or ask for help to understand. But you didn't confuse me. Ok and the last one, What are the principles of structured programming?
- Mandy:** Well, structured programming is based on principles or pre-established tasks like branching statements as If . . . Then . . . Else, loops, modularization which makes it easier to realize which part of a program is causing problems, and information hiding, it keeps information and during the procedure, it can't be damaged by any other program part.



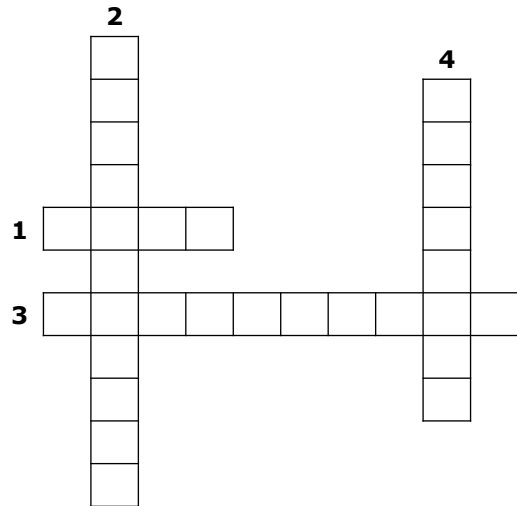
Let's Practice!

Let's clarify some concepts about software programming.

Instructions: Read the clues. Complete the crossword.

CLUES

1. Those statements made the logic of the operation more difficult to understand.
2. The structured programming refers to a set of principles or pre-established tasks and _____.
3. Structured programming has principles or pre-established tasks such as branching _____.
4. The scientist that changed programming forever was Edsger _____.



Let's Practice!

Computers weren't so complex.

Instructions: Select the correct answer.

1. Computers _____ so complex before scientists started to implement more functions.
 - a. weren't
 - b. wasn't
 - c. didn't
 - d. doesn't
2. Dijkstra stated that the "GO TO" statement _____ beneficial for programming.
 - a. wasn't
 - b. weren't
 - c. didn't
 - d. don't
3. "Go To" statements _____ easy to learn because they made the logic of the operation more difficult to understand.
 - a. weren't
 - b. didn't
 - c. wasn't
 - d. don't





Extension Activity

Structured Programming

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Search for more information about structured programming.
- Identify aspects that were not mentioned in this lesson.
- Write down a list of characteristics of structured programming.
- Share the list with the class.

Self-Assessment

Yes

No

- I search for more information about structured programming.
- I identify different aspects of structure programming.
- I write down a list of characteristics of structured programming.
- I share the list with the class.

☐ ☐
☐ ☐
☐ ☐
☐ ☐


Target Words

- **waste** (verb): not to use something to the most or to squander
e.g. I feel like I wasted my time because I didn't understand anything.
- **implement** (verb): to put into practice or introduce
e.g. When scientists started to implement more functions, computers and software became more difficult to understand.
- **beneficial** (adjective): the quality of being of good use or that brings advantage
e.g. Dijkstra stated that the "GO TO" statement wasn't beneficial for programming.
- **harmful** (adjective): something that causes damage
e.g. Those statements were harmful because they made the logic of the operation more difficult to understand.
- **vast** (adjective): something of a really big size
e.g. Sometimes you have to take plenty of notes because those topics are vast.



Learning Points

Common Auxiliaries + Not Contractions in Past: Wasn't, Weren't, Didn't

It is very common to use a wide variety of **contractions**. In the past tense, we can use **wasn't**, **weren't** and **didn't** for negative statements.

When making negative statements in simple past, we can use contractions with the **verb to be** or the auxiliary **did**. Let's check those contractions:

Was + Not	Wasn't	<i>He wasn't prepared to talk about structured programming.</i>
Were + Not	Weren't	<i>Tasks weren't easy to finish without structure programming.</i>
Did + Not	Didn't	<i>You didn't understand anything about structure planning.</i>

The use of **contractions** doesn't affect the meaning of the sentence. It is more common to use **contractions** when speaking. In formal writing, the use of **contractions** is not recommended.

Examples

- *You didn't confuse me.*
- *Dijkstra stated that the "GO TO" statement wasn't beneficial for programming.*
- *Before, computers weren't so complex.*





Learning Points

Verbs with Infinitives: Have to and Need to in Affirmative Statements

The verbs **have** and **need** express something we consider mandatory or necessary to do. It is possible to use them with **adverbs of frequency**.

We use **have to** to talk about obligations and we use **need to** to talk something that is necessary to do. To create statements, you can follow the next grammatical structure:

Subject + **Have to / Need to** + Verb + Complement

Examples:

*You **have to** study more about programming languages.*

*You **need to** pass the exam in order to get a certificate.*

We can use **adverbs of frequency** to express how often we perform an action. Take a look at the following chart to see some of the most common **adverbs of frequency**.

Adverb	Frequency
Always	100%
Usually	80%
Sometimes	50%
Rarely	20%
Never	0%

Let's use the adverbs of frequency with **need to** and **have to**.

*You **always have to** study more about programming languages.*

***Sometimes**, you **need to** ask engineers for help.*

Examples

- *Sometimes, you have to take plenty of notes because those topics are vast.*
- *I always have to make my best to explain this difficult topic.*
- *I usually need to research more or ask for help to understand.*

